AI ENGINEERING INTERNSHIP

AI Document Comparison Assistant

Project Assignment Document

Project Name	AI Document Comparison Assistant
Project Category	Generative AI / LLM Application
Required Technologies	Python, Streamlit, Gemini API or another approved LLM API, pypdf
Suggested Duration	1 week

1. Project Overview

The AI Document Comparison Assistant is a web application that allows a user to upload two PDF documents and compare their content using a Large Language Model (LLM). The application should summarize each document, identify similarities and differences, highlight important facts such as dates and requirements, and produce a final comparison report.

The project must do more than send the same prompt to an AI chatbot. Students must extract text from PDFs, validate inputs, organize the comparison results, present them clearly in a Streamlit interface, and allow the user to download the final report.

2. Problem Statement

People often need to compare two versions of a policy, course syllabus, project specification, agreement, instruction manual, or other text-based document. Reading both documents line by line can take time, and important changes may be missed. Build an application that helps users understand what is common, what has changed, and which details require attention. The AI must base its comparison on the extracted document text and must clearly indicate when information is missing or uncertain.

3. Project Objectives

- Upload and read two PDF documents.
- Generate a separate summary for each document.
- Identify key similarities and meaningful differences.
- Compare dates, names, amounts, conditions, requirements, and other important details when present.
- Display results in clearly labelled sections in a web interface.
- Generate a downloadable comparison report.
- Handle invalid files, empty documents, long documents, and API errors gracefully.

4. Intended Users and Use Cases

Intended users may include students, trainers, HR staff, administrators, and office teams.

- Compare an old and updated course syllabus.
- Compare two versions of a policy or procedure document.
- Compare two project requirement documents.
- Compare two product or service descriptions.
- Review differences between two instruction manuals.

Scope note: This is an informational comparison tool. It is not a legal decision-maker and must not claim that a document is legally safe, compliant, or equivalent.

5. Required Technology Stack

Technology	Purpose	Requirement
Python	Main application logic, text handling, validation	Required
Streamlit	Web interface and user interaction	Required
LLM API (Gemini or approved equivalent)	Summaries and document comparison	Required
pypdf	Extract text from PDF files	Required
python-dotenv	Load API key from environment variables	Required
python-docx or ReportLab	Create a downloadable DOCX or PDF report	Choose one
Git and GitHub	Version control and source-code submission	Required

6. Functional Requirements

FR-01: Upload documents — The user must be able to upload two PDF files and see their file names.

FR-02: Validate files — Reject unsupported file types, empty files, and PDFs from which no readable text can be extracted. Display a helpful message.

FR-03: Extract text — Extract text from both PDFs and keep each document's text separate. If practical, retain page numbers for traceability.

FR-04: Preview extracted content — Show a short preview or the extracted character/page count so the user can confirm the files were read.

FR-05: Summarize each document — Generate a separate summary for Document A and Document B.

FR-06: Compare similarities — List topics, statements, requirements, or facts that appear in both documents.

FR-07: Compare differences — List meaningful additions, removals, changes, or contradictions. Do not report a difference unless it is supported by the extracted text.

FR-08: Highlight key details — Where present, compare dates, names, numbers, deadlines, prices, eligibility criteria, and requirements.

FR-09: Display results — Organize the output into labelled sections or tabs: Summary A, Summary B, Similarities, Differences, Key Details, and Overall Conclusion.

FR-10: Download report — Allow the user to download the comparison result as a DOCX or PDF report.

FR-11: Error handling — Show useful messages for API failures, timeouts, overly long documents, and missing inputs.

FR-12: Reset and compare again — Allow users to clear the current comparison and upload another pair of documents.

7. Expected Application Workflow

1. User opens the Streamlit application.
2. User uploads Document A and Document B in PDF format.
3. Application validates both files and extracts their text.
4. Application checks whether usable text was extracted and whether the content fits the model's input limits.
5. Application generates a summary for each document.
6. Application sends the relevant text to the LLM with a structured comparison instruction.
7. The LLM returns similarities, differences, key details, and a conclusion.
8. Application displays the results in separate sections.
9. User downloads the final comparison report or starts a new comparison.

8. Expected User Interface

- Application title: AI Document Comparison Assistant.
- Short instruction explaining what types of documents can be compared.
- Two upload areas labelled Document A and Document B.
- A button labelled Compare Documents.
- A loading indicator while documents are being processed.
- Separate sections or tabs for both summaries, similarities, differences, key details, and conclusion.
- A Download Comparison Report button.
- Clear validation and error messages.

9. Expected Output Format

The comparison screen and downloaded report must include the following sections:

- Document A — file name and summary.
- Document B — file name and summary.
- Similarities — points supported by both documents.
- Differences — changed, added, removed, or conflicting information.
- Key Details Comparison — dates, amounts, names, requirements, and other relevant fields found in the documents.
- Overall Conclusion — a short, neutral explanation of the main changes.
- Limitations — a note if either document had unreadable pages, missing information, or content that could not be processed.

10. LLM Prompt and Response Requirements

Students must design a prompt that instructs the LLM to:

- Use only the supplied document text as evidence for the comparison.
- Keep facts from Document A separate from facts from Document B.
- Identify similarities and differences without inventing details.
- Quote or reference short supporting excerpts where useful.
- State 'Not found in the provided text' when a detail is absent.
- Return results in a predictable structure, preferably JSON or clearly labelled sections.

The Python application should validate the model response before displaying or exporting it. If JSON is used, handle malformed JSON and provide a safe error message or retry path.

11. Suggested Project Folder Structure

```
ai-document-comparison-assistant/
├── app.py
├── document_reader.py
├── comparison.py
├── report_generator.py
├── requirements.txt
├── .env.example
├── .gitignore
├── README.md
└── sample_documents/
```

```
├── document_a.pdf
└── document_b.pdf
```

Students may use a different structure if it is clear and modular. The purpose is to separate the interface, PDF text extraction, LLM comparison, and report generation rather than putting every function into one large file.

12. Testing Requirements

Test case	Expected behaviour
Two valid text-based PDFs	Summaries and comparison are generated.
One file missing	Application asks the user to upload both files.
Wrong file type	Application rejects the file with a clear message.
Scanned/image-only PDF	Application explains that no readable text was found; OCR may be added as an enhancement.
One empty or unreadable PDF	Application does not proceed with a misleading comparison.
LLM API failure	Application displays an error and lets the user retry.
Documents with different dates or numbers	The output identifies the differences accurately.
Documents with mostly identical content	The output does not invent differences.
Long document	Application handles input limits with a clear message or chunking strategy.
Download report	Downloaded report contains the displayed comparison sections.

13. Project Completion Criteria

The project is considered complete when a user can upload two valid PDFs, receive separate summaries, view similarities and differences supported by the document text, review important details, download a report, and use the application without it crashing on common invalid inputs. Students must be able to explain the PDF processing, LLM prompt, application workflow, error handling, and limitations during the final viva.

XPERT CHAMPS | AI ENGINEERING INTERNSHIP